

Absolutely Nothing to see here

Write-Up by @PHPCore

Research

Looking at the challenge there is no description and no attachment, so it most probably has something to do with the challenge site itself. My first guess was, that the flag is hidden as plain-text in the website HTML, CSS or JavaScript. So I opened up Dev-Tools and tried to search for CSCG, with no hits except for some useless snippets. I then guessed that it might be hidden in the web-requests. So I switched into the Network tab in Dev-Tools and looked at every request. Maybe it's in the request or response headers? No, maybe there are some weird requests custom to this challenge-site? Also no. Then I tried to look at the response of the GetTasks-Endpoint to ensure that there is nothing hidden inside there. But there was also nothing. So I was clueless at this point. I thought: This can't be. Then I had the idea to check out [Wayback-Machine](#) (web-archive). Maybe there was a historic version of the site, which contains the flag. But also this was a dead end. I also tried to search on the linked social-media sites and on Google, because I thought that this might be an OSINT-Challenge, which didn't lead me to the result. I then looked up ways to hide informations on a Website, which is where I found the so called **Zero-Width Characters**. I then opened up the website with Dev-Tools again, saved the tag, which typically contains the description of the challenge, as a temp-variable in the dev-console. And looked at the empty `temp0.innerText` variable. Then I checked `temp0.innerText.length`, which resulted in `146`, which gave me the idea for the following solution:

Solution

After I realized that this challenged tricked me into believing that the description was empty, when it indeed was not, I needed a way to extract the information from the tag, which is why I executed the following snippet:

```
temp0.innerText.split('').map(x => x.charCodeAt(0)).toString()
```

```
"56128,56627,56128,56643,56128,56627,56128,56631,56128,56683,56128,56665,56128,56662,56128,56592,56128,56681,56128,56671,56128,56677,56128,56592,56128,56668,56128,56671,56128,56671,56128,56667,56128,56592,56128,56668,56128,56671,56128,56670,56128,56663,56128,56592,56128,56665,56128,56670,56128,56676,56128,56671,56128,56592,56128,56676,56128,56664,56128,56661,56128,56592,56128,56678,56128,56671,56128,56665,56128,56660,56128,56604,56128,56592,56128,56676,56128,56664,56128,56661,56128,56592,56128,56678,56128,56671,56128,56665,56128,56660,56128,56592,56128,56675,56128,56676,56128,56657,56128,56674,56128,56676,56128,56675,56128,56592,56128,56676,56128,56671,56128,56592,56128,56668,56128,56671,56128,56671,56128,56667,56128,56592,56128,56658,56128,56657,56128,56659,56128,56667,56128,56592,56128,56657,56128,56676,56128,56592,56128,56681,56128,56671,56128,56677,56128,56685"
```

When we look at the sequence, we see that every other integer is the same, which is why I thought to myself: this might be useless information, so I first trimmed this away using Python:

```
' , '.join(data.split(', ')[1::2])
```

```
"56627,56643,56627,56631,56683,56665,56662,56592,56681,56671,56677,56592,56668,56671,56671,56667,56592,56668,56671,56670,56663,56592,56665,56670,56676,56671,56592,56676,56664,56661,56592,56678,56671,56665,56660,56604,56592,56676,56664,56661,56592,56678,56671,56665,56660,56592,56675,56676,56657,56674,56676,56675,56592,56676,56671,56592,56668,56671,56671,56667,56592,56658,56657,56659,56667,56592,56657,56676,56592,56681,56671,56677,56685"
```

Now we have a list of integers, with a reasonable length, but we still don't know how this was encoded, but we know that most flags start with either `CSCG{` or `dach2025{`, so I looked at the difference between the first integers and saw the following:

- 56627 -> 56643 is the same difference as 'C' -> 'S'
- 56643 -> 56627 is the same difference as 'S' -> 'C'
- 56627 -> 56631 is the same difference as 'C' -> 'G'

We now saw that the numbers are just ascii-characters with a static offset, which I converted back to ascii using the following snippet, assuming that `C -> 56627`:

```
''.join(
    map(
        lambda x: chr(int(x) - 56627 + ord('C')),
        trimmed_list.split(',')
    )
)
```

```
'CSCG{if you look long into the void, the void starts to look back at you}'
```

Now we have the flag and can enter it into the CTF-Platform.

Patching the Vulnerability

There is no real vulnerability in here. This was more of a riddle, to get to the flag. But we might assume that someone tried to hide information on his website, which everyone should be able to see, without knowing the content. With this assumption we get a well-known pit-hole in IT-Security, where people tend to believe that Encoding = Encryption, but as we have shown here, even knowing just a bit of information of the original text, tends to break Security by Obscurity systems. If we want to be able to do the same without anyone being able to see the contents, we should use real encryption algorithms like AES or RSA.